

fMRI BOLD Signal Classification

Visual Stimuli Categories from BOLD5000

Camilla Tomasoni

Università degli Studi di Milano-Bicocca
MSc Artificial Intelligence for Science and Technology

Motivation

- How does the brain encode visual information?
- Is it possible to reproduce it artificially?
- Can we decode visual categories from BOLD signals?
- Datasets used: BOLD5000 for the visual stimuli and ds001499 (Open Neuro dataset) for the fMRI data.

BOLD5000 Dataset & Compatibility Issues

BOLD5000 Dataset

- 5000 natural images
- 3 datasets: COCO, ImageNet, SUN

Dataset Compatibility Issues

- **COCO**: multi-label images
- **ImageNet**: fine-grained categories
- **SUN**: scene-only stimuli

Final labels: *object, animal or human, plant or food, scene*

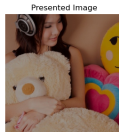
ImageNet - Category: affenpinscher, monkey pinscher, monkey dog (n02110627)
Presented



Original Image

COCO - Categories: person, teddy bear

Image with Bounding Boxes



SUN - Category: churchInside

Original

Presented



Experimental Design & Regions of Interest

Experimental Design

- **Participants:** 4 subjects
- **Sessions:** 15 sessions for subjects 1–3, 10 for subject 4
- **Runs:** Multiple runs per session
- **Trials:** 37 image trials per run
- **Timing:** 1 s image + 9 s fixation (slow event-related design)

Regions of Interest (ROIs)

- EarlyVis, LOC, OPA, PPA, RSC
- Defined using functional localizers (subtraction method)
- Include early visual cortex and high-level scene/object-selective areas

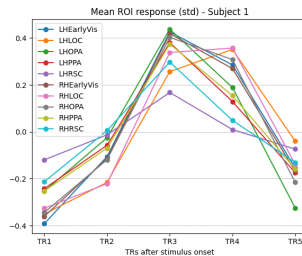
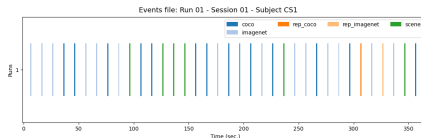


Figure: Hemodynamic Response Function (HRF): shows the delayed BOLD peak following neural activity.

Label Mapping – ImageNet

Goal: reduce the 958 fine-grained ImageNet categories to the four final labels.

- Each ImageNet class is linked to a **WordNet synset**.
- Step 1: use the **hypernym hierarchy** to obtain more abstract categories. Tested multiple generalization levels → level 6 chosen as the best trade-off between abstraction and detail.
- Step 2: embed the generalized categories using **SentenceTransformer all-MiniLM-L6-v2**.
- Created **text prototypes** for the four final labels (e.g. 'a photo of a cat' instead of 'cat').
- Assigned each ImageNet category to the closest prototype using **cosine similarity**.

Final result: a decent mapping of all ImageNet classes into the four semantic groups.

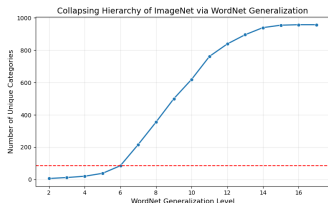


Figure: Number of ImageNet categories at different WordNet generalization levels.

Level	Synset
0	entity.n.01
1	abstraction.n.06
2	relation.n.01
3	part.n.01
4	substance.n.01
5	material.n.01
6	paper.n.01
7	tissue.n.02
8	toilet_tissue.n.01

Table: Example hypernym chain for *toilet_tissue*.

Label Mapping – COCO

Goal: assign each COCO category to one of the four final labels.

- COCO is **multi-label**: each image contains several objects → difficult to assign a single label.
- First idea: for each image find the most common label in it and discard the others, then use **sentence embeddings** on single-word labels → unreliable results.
- Poor results: the multi-label nature of COCO remains incompatible with single-label classification.

Final decision: COCO was excluded from the fMRI analysis due to its multi-label structure.

Label Mapping – SUN

Goal: map SUN categories into the four final labels.

- SUN contains only **scene categories** (indoor and outdoor environments).
- First attempt: map SUN labels using **sentence embeddings** → unsuccessful due to semantic mismatch.
- Second attempt: **clustering** the embedded labels → clusters still represented only scene types.
Method used: all-MiniLM-L6-v2 SentenceTransformer to compute embeddings for the SUN labels, and Agglomerative Clustering (n=10) to identify semantic groups.
- SUN categories do not naturally fit into *object*, *animal* or *human*, or *plant* or *food*.

Final decision: assign **all SUN categories** to the label *scene*.

Topic 1	Topic 2
factory, sushibar, frontFoyer, garbagedump, bath-roomSink, frontLawn, pizzeria, ...	office, MRI, pharmacy, sculpturegarden, homeoffice, flowershop, nursery, ...

Table: Example clusters obtained from SUN label embeddings. All clusters still represent **scene categories**, confirming that SUN cannot be mapped to object-, animal-, or food-related labels.

Final Labels Distribution

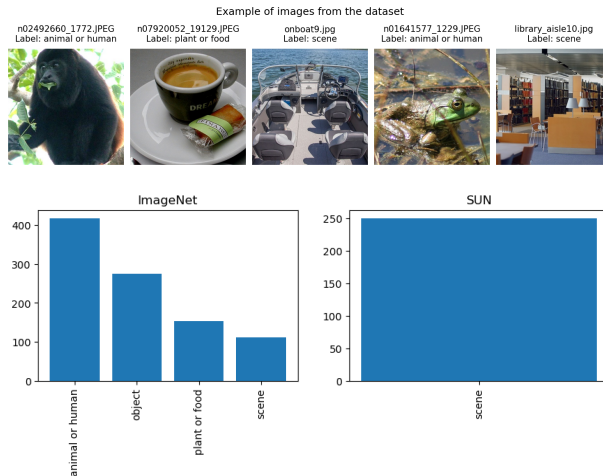


Figure: Above: Examples of images and their final assigned label. Below: Final labels distribution.

Why Extract Voxel Responses?

Problem: fMRI data are 4D volumes (x, y, z, time). Each run contains hundreds of whole-brain volumes, but the classification task requires **one feature vector per trial**.

- Raw BOLD time series are **not directly usable** since each voxel contains a full time course.
- We need a method that isolates the response to **each individual stimulus**.
- Solution: estimate a **beta map per trial**, representing voxel-wise activation specific to that stimulus.

Goal: convert 4D fMRI runs into **clean, comparable feature vectors** for machine learning.

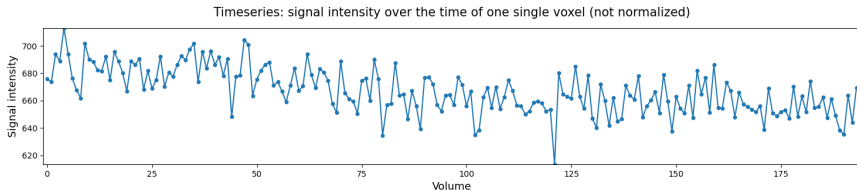


Figure: Example of a raw BOLD time series: intensity of one voxel of a specific subject over time, in one run.

Voxel Response Extraction: GLM + LSS

General Linear Model (GLM): A statistical model that explains the BOLD signal as a weighted combination of predictors:

$$\text{BOLD}(t) = X\beta + \epsilon$$

- X : design matrix (stimulus regressors + confounds)
- β : weights estimating voxel response strength
- ϵ : noise

Least-Squares-Separate (LSS): A GLM variant where we fit **one small GLM per trial**:

- one regressor for the target trial,
- one regressor grouping all other trials.

Why LSS?

- It allows to isolate the brain activity caused *only* by the image shown in that trial.
- It gives one clean activation map per stimulus, making it possible to compare trials and train a classifier.

MLP on Whole-Brain LSS Voxel Features

Input Representation

- Each trial is represented by a **whole-brain voxel vector** extracted from LSS beta maps.
- No ROI masking applied: all voxels are included, but vectors differ in length across subjects.
- Applied **PCA** to project each vector into a fixed **1024-dimensional** space.

- Fully connected MLP

- Grid search over 288 configurations.

- Best model:

- activation = logistic
- solver = Adam
- $\alpha = 0.01$
- batch size = 32
- hidden layers = [256, 64]

Hyperparameter	Values Tested
Hidden layers	(512,256,128,64), (256,128,64), (256,64), (128,64)
Activation	relu, logistic, tanh
Solver	sgd, adam
α	1e-5, 1e-4, 1e-3, 1e-2
Batch size	32, 64, 128
Total combinations	288

Results: Accuracy = 0.44, Precision = 0.26, Recall = 0.25, F1-score = 0.25 — performance close to **random guessing**, likely due to class imbalance and unsuitable model to the biological context.

Voxel-to-Feature Mapping: Encoding Approach

Goal: map fMRI voxel responses into the same feature space used to represent images.

- Each stimulus is encoded using a deep visual model (ResNet-50 or a custom CNN).
- The resulting feature vector (e.g., 2048-D from ResNet) represents the image in a high-level semantic space.
- For each voxel, we fit a voxel-wise encoding model:
 - input: image feature vector,
 - output: voxel activation (beta value).
- Voxels with higher predictive correlation are selected within each ROI to reduce noise and prevent overfitting.
- The selected voxel responses form the input to hierarchical decoding models (BRNN, GNN, multi-branch GNN, Transformer).

Intuition: learn how each voxel responds to visual features, then use this mapping to decode the stimulus from brain activity.

Visual Encoding: Custom CNN

Grid Search Summary

- 16 total combinations
- Best params (macro-F1 on validation):
 - lr = 0.001
 - optimizer = adam
 - num_filters = 32
 - batch size = 64
- Final model trained for **100 epochs**

Model Architecture

- 2 convolutional blocks (Conv + BN + ReLU + MaxPool)
- Adaptive average pooling to fixed 4×4
- 2 fully connected layers (32-dim embedding → classifier)
- Dropout for regularization

Hyperparameter	Values Tested
Learning rate	0.01, 0.001
Optimizer	adam, sgd
Num filters	16, 32
Batch size	32, 64
Total combinations	16

Table: Grid search hyperparameters.

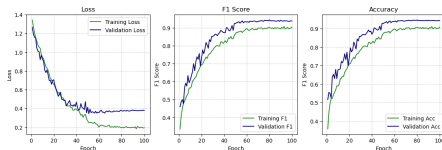


Figure: Simple CNN training.

Visual Encoding: Fine-Tuned ResNet-50

Approach

- Started from a **pre-trained ResNet-50** (ImageNet).
- All convolutional layers frozen** to preserve pretrained visual features.
- Replaced the final fully connected layer with:

`Linear(2048 → num_classes)`

- Implemented a custom `extract_features()` function to obtain the 2048-D embedding before the classifier.
- Only the **last classification layer** was trained.

Training Setup: Trained using the same best hyperparameters found for the custom CNN. Trained for **40 epochs**.

Outcome: Obtained a 2048-D feature, instead of the 32-D feature obtained from the custom CNN.

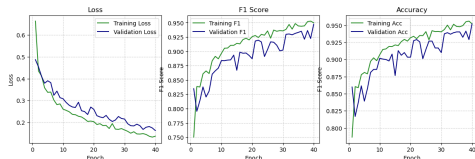


Figure: ResNet-50 Fine-Tuning.

ResNet-50 vs Custom CNN: Comparison

	Fine-Tuned ResNet-50	Custom CNN
Pretraining	ImageNet (deep, rich features)	None (trained from scratch)
Trainable Params	Only final FC layer	Entire network
Feature Dim.	2048-D embedding	32-D embedding
Epochs	30 (benefits from pretraining)	100 (needs more training)
Test Accuracy	0.950	0.954
Test F1 Score	0.942	0.948

Table: Comparison of the two visual encoding strategies.

Since the results are the same, I chose to use the ResNet-50 as feature extractor because:

- it stays faithful to the methodology proposed in the reference paper;
- it provides higher-dimensional feature representations (2048-D), which are likely to be more informative and semantically rich.

From Image Features to Voxels: Selection via Regression

Goal

- Identify the **most informative voxels** in each ROI (or ROI pair), that is, the most correlated with the image features.
- These voxels will form fixed-size inputs for the decoding models (BRNN, GNN, multi-branch GNN).

Voxel-wise Encoding via Ridge Regression

- For each ROI (or ROI pair), we build an encoding model:

$$X \in \mathbb{R}^{n_{\text{trials}} \times n_{\text{feat}}} \rightarrow Y \in \mathbb{R}^{n_{\text{trials}} \times n_{\text{voxels}}}$$

where X are CNN features and Y are voxel responses.

- **multi-output Ridge Regression** (L2-regularized linear regression)
- α is chosen with **RidgeCV** on a random subset (500 samples), maximizing the average R^2 across folds.

Selecting Top-k Voxels

- Using the best α , fitting of Ridge Regression on train data and predict voxel responses on validation data.
- For each voxel, compute the **Pearson correlation** between predicted and true responses.
- Voxels are **ranked** by this correlation score, and the **top-k** are kept as the most informative.

ROI-based Voxel Selection for Hierarchical Approaches

Masking Strategy: Ridge-based voxel selection is applied **independently per ROI (or ROI pair)**. This yields a representation for each region, derived from CNN/ResNet features.

BRNN: ROI Pairs as Time Steps

- For the BRNN, the input is a **sequence of ROI pairs**: (LHEarlyVis, RHEarlyVis), (LHLOC, RHLOC), ...
- For each ROI pair, **top 100 voxels** are selected using the Ridge + correlation procedure.
- Each BRNN time step thus receives a fixed-length vector (100 voxels per ROI pair), respecting the hierarchical order of the ventral visual stream.

Multi-Branch GNN: Separate Hemispheres

- For the multi-branch GNN, left and right hemispheres are modeled by **two separate branches**. therefore **single ROIs** are used instead of ROI pairs.
- For each ROI, **51 voxels** are selected, so that each branch processes its own ROI sequence with a fixed input size per node.

Resulting Representation

- This procedure produces **fixed-size, ROI-structured voxel vectors**, aligned with image features via regression, and ready to be used as input for BRNN and GNN decoding models.

Hierarchical Approach

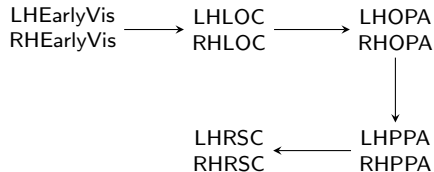
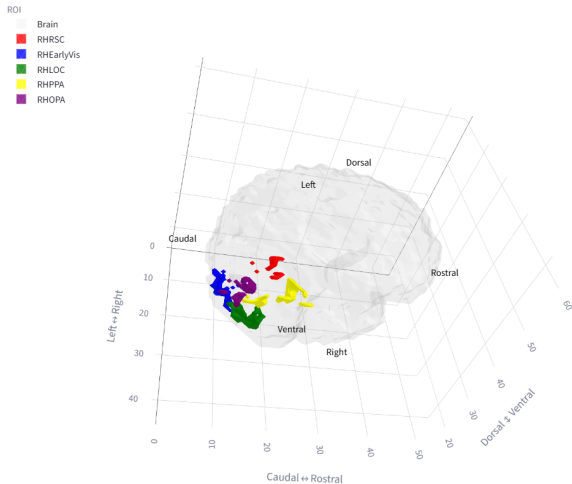


Figure: Sequence of ROI pairs used as BRNN and GNN time steps.

Figure: ROIs of the right hemisphere in subject CSI1.

BRNN and LSTM: Intuition

Bidirectional Recurrent Neural Network (BRNN)

- Processes the input sequence in two directions:
 - **forward:** EarlyVis $\rightarrow$ LOC $\rightarrow$ OPA $\rightarrow$ PPA $\rightarrow$ RSC
 - **backward:** RSC $\rightarrow$ PPA $\rightarrow$ OPA $\rightarrow$ LOC $\rightarrow$ EarlyVis
- Each timestep has access to both past and future context.
- In this task, each ROI pair benefits from information coming from the entire visual hierarchy.

Long Short-Term Memory (LSTM)

- A recurrent unit designed to capture long-range dependencies.
- Uses three gates:
 - **input gate:** how much new information enters the memory
 - **forget gate:** what information to discard
 - **output gate:** what part of the memory is exposed
- Avoids vanishing gradients and models structured temporal relationships.
- In our case, it learns how different ROIs contribute to scene-category decoding.

BRNN Model Overview

Input Representation

- Each trial is encoded as a **sequence of 5 ROI pairs**.
- Each ROI pair provides a fixed-size vector of selected voxels (100 voxels).
- Final input shape per trial: sequence length = 5, feature size = 100

Model Structure

- Bidirectional LSTM with:
 - input size = 100
 - hidden size = `hidden_size`
 - number of layers = `num_layers`
 - `bidirectional = True` (forward + backward)
- Dropout applied to LSTM outputs.
- Fully connected layer:
$$\text{Linear}(2 \cdot \text{hidden_size} \rightarrow \text{num_classes})$$
- Output: class prediction from the last timestep (concatenated forward + backward states).

BRNN Hyperparameter Search

Hyperparameter	Values Tested
Normalization	z_score, min_max
Hidden size	16, 32
Number of layers	1, 2
Dropout	0.5
Batch size	32, 64
Optimizer	Adam, SGD
Learning rate	0.001, 0.0001
Total combinations	64

Table: Hyperparameter grid used for the BRNN model search.

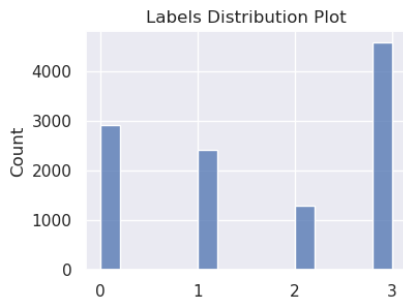


Figure: Distribution of the dataset used in all decoding models.

The best-performing configuration was selected based on validation Macro F1 Score, using the same train+val/test split across all decoding models. Best parameters: batch_size = 32, hidden_size = 32, learning rate = 0.001, normalization = z_score, num_layers = 1, optimizer = Adam.

BRNN Results and Analysis

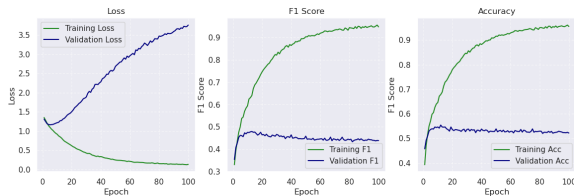


Figure: Training curves of BRNN.

- Train sequences: (8954, 5, 100)
- Test sequences: (2239, 5, 100)

Test Performance

- Accuracy: **0.34**
- Macro F1-score: **0.28**
- Precision: **0.30**
- Recall: **0.29**

Considerations

- The rationale behind the BRNN seems solid, since it effectively exploits the sequential structure of the ROI pairs, capturing both bottom-up and top-down interactions across the visual hierarchy.
- Class imbalance has been handled with class weights in input to the Cross Entropy Loss function.
- Poor results, just slightly better than random guessing. The metrics on training set actually improve during the epochs, but not on the validation. It suggests that there is a problem of generalization, which could be given by too few data (see slide 'Challenges').

Graph Neural Networks: Rationale

Why Graph Neural Networks?

- BRNN treated ROIs as **sequences**, assuming a fixed hierarchical order. Idea: consider brain regions as a network of interconnected areas.
- A Graph Neural Network (GNN) models the brain as a **graph**, where:
 - nodes = ROIs
 - edges = functional/structural connectivity
- This allows the model to exploit **inter-ROI relationships** rather than a fixed temporal structure.
- GAT layers assign **attention weights** to neighbors, focusing on the most informative connections.

Multibranch GNN: Motivation

- A standard GNN merges all ROIs into a single graph, mixing left and right hemispheres.
- The Multibranch GNN introduces **two parallel branches**:
 - one GAT-based encoder for the **Left Hemisphere (LH)**
 - one GAT-based encoder for the **Right Hemisphere (RH)**
- Each branch produces a hemisphere-specific embedding; the two embeddings are then **concatenated** and fed to a classifier.

Multibranch GNN: Architecture and Hyperparameter Search

Model Architecture

- The model consists of two parallel GAT-based branches, for the two hemispheres.
- Each branch applies:
 - One or two GATConv layers
 - BatchNorm $\rightarrow$ ReLU $\rightarrow$ Dropout
 - Global mean pooling to obtain a hemisphere embedding
- The two hemisphere embeddings are concatenated.
- Final classifier:

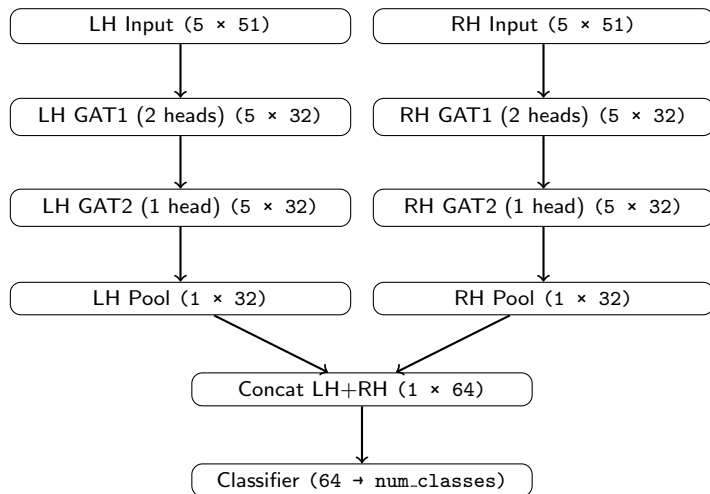
Linear(emb $\rightarrow$ num_classes)

Best configuration (Macro F1): batch size = 32, hidden channels = 16, learning rate = 0.001, normalization = z_score, num layers = 2, optimizer = Adam.

Hyperparameter	Values Tested
Normalization	z_score, min_max
Hidden channels	8, 16
Number of layers	1, 2
Dropout	0.5
Batch size	32, 64
Optimizer	Adam, SGD
Learning rate	0.001, 0.0001
Total combinations	64

Table: Hyperparameter grid used for the Multibranch GNN.

Multibranch GNN: Compact Architecture Diagram



Training Approach

Dataset Split: Identical to BRNN.

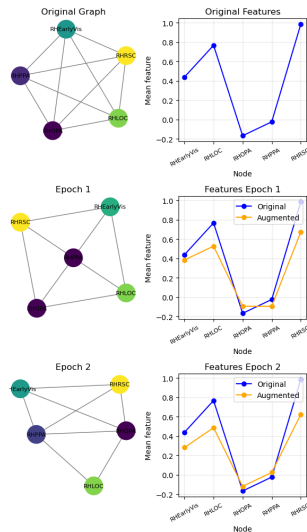
Training Steps:

3 Multibranch GNN models with best parameters (from grid search), all trained for 150 epochs.

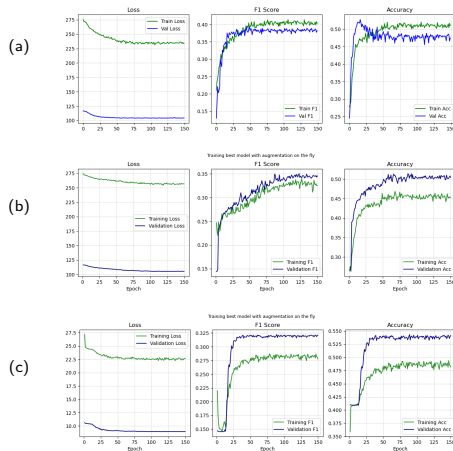
- Weighted Cross Entropy loss, No augmentation
- Weighted Cross Entropy loss, Augmentation on-the-fly
- Focal Loss ($\gamma = 8.0$), Augmentation on-the-fly

Graph Augmentation (on-the-fly)

1. Edge dropout ($p=0.4$)
2. Feature masking ($p=0.3$)
3. Gaussian noise ($\text{std}=0.02$)

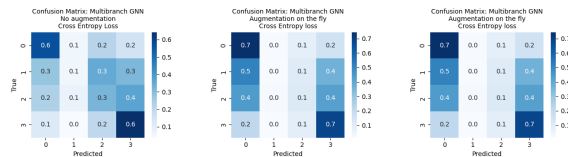


Multibranch GNN Train and Test



Loss	Aug	Test Acc	Test F1	Test Prec	Test Rec
CE	No	0.49	0.39	0.42	0.43
CE	Yes	0.51	0.35	0.36	0.40
Focal	Yes	0.54	0.32	0.27	0.40

Table: Multibranch GNN test performance across training setups.



Considerations: All the three models struggle to assign samples to classes 1 and 2, since they are the less frequent ones. Model performed better without augmentation, which is probably too aggressive and reduces the significative differences between the signals, needed to distinguish them.

Recap & Challenges

Steps

Common labels (SentenceTransformer, WordNet)

Visual Encoding (ResNet50, custom CNN + Ridge Regression)

Visual Decoding (BRNN, GNN)

Models ranking based on Test Macro F1-Score

Multibranch GNN (CE, no aug) — $F1 = 0.39$, $Acc = 0.49$

Multibranch GNN (CE, aug) — $F1 = 0.35$, $Acc = 0.51$

Multibranch GNN (FL, aug) — $F1 = 0.32$, $Acc = 0.54$

BRNN — $F1 = 0.28$, $Acc = 0.34$

Challenges

Voxel extraction strategy

Class imbalance difficult to address

Assumption about ROI ordering

Use of paired ROIs (BRNN)

Dataset size

Future Directions

Voxel selection (performance comparison by changing k)

Different voxel extraction strategy

Different visual encoding strategy (low-dimensional features instead of 2048-D)

References

- Chang, N., Pyles, J. A., Marcus, A., Gupta, A., Tarr, M. J., & Aminoff, E. M. *BOLD5000, a public fMRI dataset while viewing 5000 visual images*. Scientific Data, 6(49), 2019.
- Miyawaki, Y., Uchida, H., Yamashita, O., Sato, M., Morito, Y., Tanabe, H. C., Sadato, N., & Kamitani, Y. *Visual image reconstruction from human brain activity using a combination of multiscale local image decoders*. Neuron, 60(5), 915–929, 2008. doi:10.1016/j.neuron.2008.11.004
- Qiao, K., Chen, J., Wang, L., Zhang, C., Zeng, L., Tong, L., & Yan, B. *Category Decoding of Visual Stimuli From Human Brain Activity Using a Bidirectional Recurrent Neural Network*. Frontiers in Neuroscience, 13:692, 2019. doi:10.3389/fnins.2019.00692
- Sentence-Transformers Team. *all-MiniLM-L6-v2*. <https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2> (Accessed: 2025-02-14)
- amikeline. *COCO Labels 2014/2017*. https://github.com/amikeline/coco-labels/blob/master/coco-labels-2014_2017.txt (Accessed: 2025-02-14)
- Ayoosh Kathuria. *Understanding Bidirectional RNN in PyTorch*. <https://medium.com/data-science/understanding-bidirectional-rnn-in-pytorch-5bd25a5dd66> (Accessed: 2025-02-14)